

Video RAG Project - Source Code

File Name: app.py | Language: Python (Streamlit, Whisper, LangChain, ChromaDB, Groq)

Application Source Code (app.py)

```
import os
import streamlit as st
import whisper
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.vectorstores import Chroma
from groq import Groq

# Custom Styling
st.markdown(
    """
    <style>
    .main { background-color: #0e1117; }
    .stButton>button { background-color: #ff4b4b; color: white; border-radius: 8px; font-weight: bold; }
    .stTextInput>div>div>input { border-radius: 8px; }
    </style>
    """,
    unsafe_allow_html=True,
)

st.title("Video RAG Project - Transcript Extractor")

uploaded_file = st.file_uploader("Upload your video file", type=["mp4", "mov", "avi"])

if uploaded_file is not None:
    with open("temp_video.mp4", "wb") as f:
        f.write(uploaded_file.getbuffer())

    if st.button("Transcribe Video"):
        with st.spinner("Transcribing video using Whisper..."):
            model = whisper.load_model("base")
            result = model.transcribe("temp_video.mp4")
            transcript_text = result["text"]
            st.success("Video Transcribed Successfully!")
            st.subheader("Transcript:")
            st.write(transcript_text)

            text_splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
            chunks = text_splitter.split_text(transcript_text)

            embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
            vectorstore = Chroma.from_texts(chunks, embeddings, persist_directory="./chroma_db")
            st.success("Transcript successfully stored in Vector Database (ChromaDB)!")

            st.subheader("Database Chunks Preview:")
            for i, chunk in enumerate(chunks[:3]):
                st.write(f"***Chunk {i+1}:** {chunk}")

    st.subheader("Ask a Question from Videos")
    user_query = st.text_input("Apni video se related sawal yahan likhein:")

    if user_query:
        if os.path.exists("./chroma_db"):
            embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
```

```
vectorstore = Chroma(persist_directory="./chroma_db", embedding_function=embeddings)
docs = vectorstore.similarity_search(user_query, k=2)

if docs:
    context = "\n".join([doc.page_content for doc in docs])

    client = Groq(api_key=os.environ.get("GROQ_API_KEY"))
    prompt = f"Answer the user query based strictly on the context provided below:\nContext:\n{context}"

    chat_completion = client.chat.completions.create(
        messages=[{"role": "user", "content": prompt}],
        model="openai/gpt-oss-120b",
    )
    answer = chat_completion.choices[0].message.content
    st.subheader("Answer:")
    st.success(answer)
else:
    st.warning("Relevant data not found.")
else:
    st.warning("Please upload and transcribe a video first.")
```